

Modelos probabilísticos de lenguaje

Julian D. Echeverry-Correa
jde@utp.edu.co

Técnicas de Procesamiento de Lenguaje Natural

Doctorado en Ingeniería - Maestría en Ingeniería Eléctrica
Maestría en Ingeniería de Sistemas - Maestría en Investigación Operativa y Estadística

Grupo de Investigación en Análisis de Datos y Sociología Computacional - GIADSc - 2025



Contenido

- 1 Introducción
- 2 Modelos de Lenguaje
- 3 Estimación estadística de LM
- 4 Evaluación de LMs
- 5 Tarea - Un primer modelo de lenguaje
- 6 Herramientas

- 1 Introducción
- 2 Modelos de Lenguaje
- 3 Estimación estadística de LM
- 4 Evaluación de LMs
- 5 Tarea - Un primer modelo de lenguaje
- 6 Herramientas

- Los modelos de lenguaje son sistemas que asignan una probabilidad a una secuencia de palabras. Esta probabilidad se utiliza para diversas aplicaciones, como la predicción de texto, la traducción automática, el reconocimiento de voz y la corrección ortográfica.
- Históricamente, los modelos de lenguaje han evolucionado desde enfoques estadísticos clásicos hasta los modernos modelos basados en *deep learning*.
- En esta clase, nos centraremos en los enfoques probabilísticos clásicos.

- Cuando se hace procesamiento de textos, podemos escoger entre dos enfoques, que no necesariamente son antagónicos, incluso pueden ser complementarios:
 - Modelado **independiente** del contexto.
 - Modelado **dependiente** del contexto.

Veamos un ejemplo en estas frases

Ella me dijo: me gusta el olor de las flores.

Le dije a ella: me gusta tu olor a flores.

Siguiendo los pasos de un procesamiento clásico basado en la eliminación de *stop-words*, la tokenización, la lematización, etc., los textos anteriores podrían convertirse en:

Tokens de la frase 1:

ella
decir
gustar
olor
flor

Tokens de la frase 2:

ella
decir
gustar
olor
flor

El modelado independiente del contexto puede hacer que pierdan sentido las frases.

- La *inferencia estadística* consiste, básicamente, en tomar ciertos datos (generados a partir de una función de distribución de probabilidad desconocida) e inferir características de esta distribución.
- Los **modelos probabilísticos (estadísticos) de lenguaje** se basan en la misma premisa. A partir de un conjunto de datos conocidos (palabras o secuencias de palabras en un corpus) se infieren propiedades acerca de la distribución que generó dichos datos.

- Los **modelos de lenguaje** son el corazón de los sistemas de procesamiento de lenguaje.
- Son en esencia **modelos probabilísticos (estadísticos)** encargados de analizar los patrones existentes en el lenguaje.
- En los modelos de lenguaje, el objetivo es muy claro: **predecir** la siguiente palabra dada la anterior o dada una secuencia anterior de palabras. No entendamos “predecir” como un simple acto adivinatorio, sino como la asignación de probabilidades a cada una de las diferentes hipótesis.

- Existen dos grandes tipos de **modelos de lenguaje**:
 - Modelos estadísticos (basados en la estimación de probabilidades de ocurrencia).
 - n-grams
 - bidireccionales
 - exponenciales (a.k.a. de máxima entropía)
 - de espacio continuo
 - Modelos de lenguaje soportados por redes neuronales (basados en el entrenamiento a partir de grandes bases de datos).

The roots of language modeling as it exists today can be traced back to 1948. That year, Claude Shannon published a paper titled "A Mathematical Theory of Communication." In it, he detailed the use of a stochastic model called the Markov chain to create a statistical model for the sequences of letters in English text. This paper had a large impact on the [telecommunications](#) industry, laid the groundwork for [information theory](#) and language modeling. The Markov model is still used today, and n-grams specifically are tied very closely to the concept.

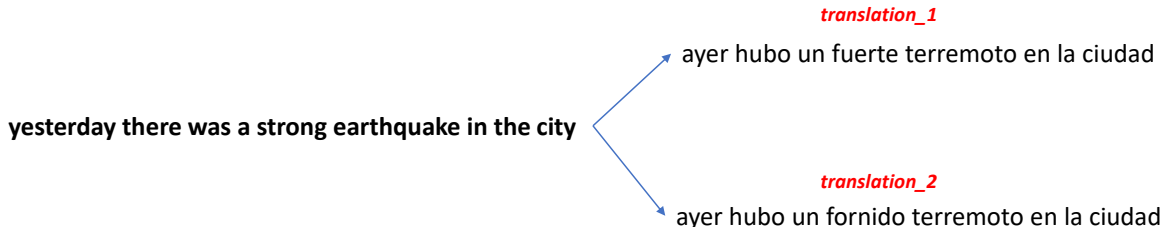
Contenido

- 1 Introducción
- 2 Modelos de Lenguaje**
- 3 Estimación estadística de LM
- 4 Evaluación de LMs
- 5 Tarea - Un primer modelo de lenguaje
- 6 Herramientas

Modelos de Lenguaje (*Language Models* - LM)

Objetivo general de los LM: Asignar **probabilidades** a **secuencias de palabras**

- ¿En qué se aplican los LM?
 - En tareas de **traducción automática**



$$p(\text{fuerte terremoto}) > p(\text{fornido terremoto})$$

Modelos de Lenguaje (*Language Models* - LM)

Objetivo general de los LM: Asignar **probabilidades** a **secuencias de palabras**

- ¿En qué se aplican los LM?
 - En tareas de **corrección ortográfica**

Me llamó la atención de que no viniera



$p(\text{Me llamó la atención que no vinera}) > p(\text{Me llamó la atención DE que no viniera})$



Me llamó la atención ~~de~~ que no viniera

Modelos de Lenguaje (*Language Models* - LM)

Objetivo general de los LM: Asignar **probabilidades** a **secuencias de palabras**

- ¿En qué se aplican los LM?
 - En tareas de **reconocimiento de habla**



Fonéticamente, ambas posibilidades pueden ser equiprobables

Me voy a *casar*
cazar , pásame la escopeta

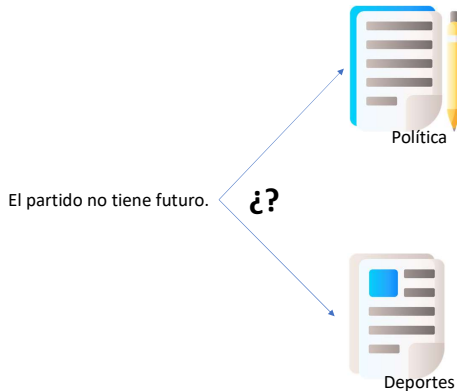
* Escenario probable en pronunciación nativa de Suramérica

$p(\text{me voy a cazar, pásame la escopeta}) > p(\text{me voy a casar, pásame la escopeta})$

Modelos de Lenguaje (*Language Models* - LM)

Objetivo general de los LM: Asignar **probabilidades** a **secuencias de palabras**

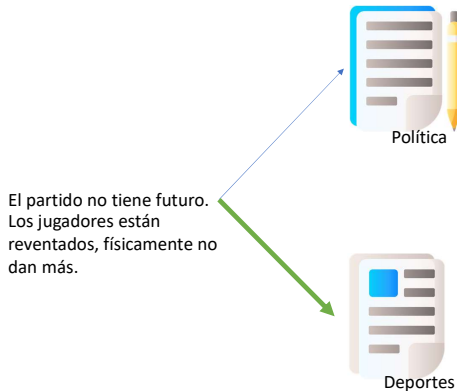
- ¿En qué se aplican los LM?
 - En tareas de **clasificación de textos e identificación de temática**



Modelos de Lenguaje (*Language Models* - LM)

Objetivo general de los LM: Asignar **probabilidades** a **secuencias de palabras**

- ¿En qué se aplican los LM?
 - En tareas de **clasificación de textos e identificación de temática**



Objetivo general de los LM: Asignar **probabilidades** a **secuencias de palabras**

- ¿En qué se aplican los LM?
 - En tareas de generación de textos, OCR - reconocimiento óptico de caracteres, generación de resúmenes, sistemas de Q&A, sistemas de diálogo, chatbots, etc.
 - Generación de modelos empleando un LM Knesser-Ney (<https://mesotron.shinyapps.io/languagemodel/>)
 - Creación de textos a partir de LM entrenados con redes neuronales (cualquier IA hoy en día hace cosas como esta)

Formalizando los Modelos de Lenguaje

- Como ya dijimos, el objetivo es calcular la probabilidad de secuencias de palabras

$$P(W) = P(w_1, w_2, w_3, \dots, w_n)$$

- Probabilidad de una palabra entrante

$$P(w_5 | w_1, w_2, w_3, w_4)$$

- Un modelo que sea capaz de calcular estas probabilidades se conoce entonces como un **modelo de lenguaje**, un **modelo gramatical** o simplemente **LM**.

- ¿Cómo calcular la siguiente probabilidad conjunta?

$$P(\text{los, motivos, que, tuvimos, para})$$

- Lo primero que se nos ocurre es emplear la regla de la cadena

$$P(a, b, c, d) = P(a)P(b|a)P(c|b, a)P(d|c, b, a)$$

- Aplicada a palabras en una secuencia

$$\begin{aligned} &P(\text{los}) \times P(\text{motivos}|\text{los}) \\ &\times P(\text{que}|\text{los motivos}) \times P(\text{tuvimos}|\text{los motivos que}) \\ &\times P(\text{para}|\text{los motivos que tuvimos para}) \end{aligned}$$

- ¿Se puede calcular simplemente contando y dividiendo?

$$P(\text{para}|\text{los motivos que tuvimos}) = \frac{\text{Conteo}(\text{los motivos que tuvimos para})}{\text{Conteo}(\text{los motivos que tuvimos})}$$

- Respuesta: ¡¡ NO !!
- Tendríamos muchísimas frases en las cuales hacer estos cálculos.
- En realidad no tendríamos suficientes datos para estimar la probabilidad de cada secuencia de palabras.

- ¿Se puede calcular simplemente contando y dividiendo?

$$P(\text{para}|\text{los motivos que tuvimos}) = \frac{\text{Conteo}(\text{los motivos que tuvimos para})}{\text{Conteo}(\text{los motivos que tuvimos})}$$

- Respuesta: ¡¡ **NO** !!
- Tendríamos muchísimas frases en las cuales hacer estos cálculos.
- En realidad no tendríamos suficientes datos para estimar la probabilidad de cada secuencia de palabras.

Asunción de Markov: Asumimos que la probabilidad de una palabra depende **solo** de la palabra previa.

- Asunción de Markov

$$P(\text{para} \mid \text{los motivos que tuvimos}) \approx P(\text{para} \mid \text{tuvimos})$$

Los modelos de Markov asumen que se puede predecir la probabilidad de un evento sin necesidad de mirar muy atrás en el pasado. En ese orden de ideas, podríamos aproximar la probabilidad mirando una palabra más hacia atrás.

$$P(\text{para} \mid \text{los motivos que tuvimos}) \approx P(\text{para} \mid \text{que tuvimos})$$

- Cuando analizamos la probabilidad de una palabra dada una palabra anterior, hablamos de un **bigrama**

$$P(\text{para} \mid \text{tuvimos})$$

- Cuando analizamos la probabilidad de una palabra dadas dos palabras anteriores, hablamos de un **trigrama**

$$P(\text{para} \mid \text{que tuvimos})$$

- Y de manera general, cuando analizamos la probabilidad de una palabra dadas **N** palabras anteriores, hablamos de un **N-grama**

La ecuación general de un N-grama para calcular la probabilidad condicional de la siguiente palabra en una secuencia es

$$P(w_n | w_1^{n-1}) \approx P(w_n | w_{n-N+1}^{n-1})$$

Por ejemplo, dada entonces la asunción de bigrama, podemos calcular la probabilidad de una secuencia de palabras como

$$P(w_1^n) = P(w_1 w_2 \dots w_n) \approx \prod_{k=1}^n P(w_k | w_{k-1})$$

Caso más simple: unigrama

- Se calcula como la probabilidad de cada palabra en el corpus.

$$P(w_1^n) = P(w_1 w_2 \dots w_n) = \prod_{k=1}^n P(w_k)$$

Caso general: *N*-gramas

- Lo mismo puede extenderse a tri-gramas (*tri-grams*), cuatri-gramas (*4-grams*), quinti-gramas (*5-grams*), etc.
- Hay que reconocer que, desde el punto de vista teórico, este modelo resulta insuficiente. El lenguaje tiene dependencias que en muchos casos, llegan a ser muy largas.

El computador, que me he comprado el fin de semana pasado, no arranca.

- A pesar de esto, los *N*-gramas dan muy buenos resultados y están en el estado del arte de los métodos estadísticos para modelar lenguaje.
- Investigar qué son los skip-grams y cómo pueden corregir estos problemas.

Caso general: *N*-gramas

- Lo mismo puede extenderse a tri-gramas (*tri-grams*), cuatri-gramas (*4-grams*), quinti-gramas (*5-grams*), etc.
- Hay que reconocer que, desde el punto de vista teórico, este modelo resulta insuficiente. El lenguaje tiene dependencias que en muchos casos, llegan a ser muy largas.

El computador, que me he comprado el fin de semana pasado, no arranca.

- A pesar de esto, los *N*-gramas dan muy buenos resultados y están en el estado del arte de los métodos estadísticos para modelar lenguaje.
- **Investigar qué son los skip-grams y cómo pueden corregir estos problemas.**

Ejemplos en python

```
In [1]: import nltk
from nltk.collocations import *
texto = 'A mí no me gustan las sopas de sobre. A mí no me gustan los jugos de caja!'
tokens = nltk.word_tokenize(texto)
list(nltk.bigrams(tokens))
```

```
Out[1]: [('A', 'mí'),
('mí', 'no'),
('no', 'me'),
('me', 'gustan'),
('gustan', 'las'),
('las', 'sopas'),
('sopas', 'de'),
('de', 'sobre'),
('sobre', '.'),
('.', 'A'),
('A', 'mí'),
('mí', 'no'),
('no', 'me'),
('me', 'gustan'),
('gustan', 'los'),
('los', 'jugos'),
('jugos', 'de'),
('de', 'caja'),
('caja', '!')]
```

Ejemplos en python

```
In [2]: bigram_measures = nltk.collocations.BigramAssocMeasures()
bigram_finder = BigramCollocationFinder.from_words(tokens)
bigram_scored = bigram_finder.score_ngrams(bigram_measures.raw_freq)
bigram_scored[:]
```

```
Out[2]: [((('A', 'mí'), 0.1),
          (('me', 'gustan'), 0.1),
          (('mí', 'no'), 0.1),
          (('no', 'me'), 0.1),
          (('.', 'A'), 0.05),
          (('caja', '!'), 0.05),
          (('de', 'caja'), 0.05),
          (('de', 'sobre'), 0.05),
          (('gustan', 'las'), 0.05),
          (('gustan', 'los'), 0.05),
          (('jugos', 'de'), 0.05),
          (('las', 'sopas'), 0.05),
          (('los', 'jugos'), 0.05),
          (('sobre', '.'), 0.05),
          (('sopas', 'de'), 0.05)]
```

Ejemplos en python

```
In [3]: list(nltk.trigrams(tokens))
```

```
Out[3]: [('A', 'mí', 'no'),  
         ('mí', 'no', 'me'),  
         ('no', 'me', 'gustan'),  
         ('me', 'gustan', 'las'),  
         ('gustan', 'las', 'sopas'),  
         ('las', 'sopas', 'de'),  
         ('sopas', 'de', 'sobre'),  
         ('de', 'sobre', '.'),  
         ('sobre', '.', 'A'),  
         ('.', 'A', 'mí'),  
         ('A', 'mí', 'no'),  
         ('mí', 'no', 'me'),  
         ('no', 'me', 'gustan'),  
         ('me', 'gustan', 'los'),  
         ('gustan', 'los', 'jugos'),  
         ('los', 'jugos', 'de'),  
         ('jugos', 'de', 'caja'),  
         ('de', 'caja', '!')]
```

Ejemplos en python

Y en el caso del cálculo de colocaciones, la sintaxis es muy similar.

```
In [4]: bigram_pmi_scored = bigram_finder.score_ngrams(bigram_measures.pmi)
bigram_pmi_scored
```

```
Out[4]: [(['caja', '!'), 4.321928094887363),
          (['las', 'sopas'], 4.321928094887363),
          (['los', 'jugos'], 4.321928094887363),
          (['sobre', '.'], 4.321928094887363),
          (['.', 'A'], 3.3219280948873626),
          (['A', 'mí'], 3.3219280948873626),
          (['de', 'caja'], 3.3219280948873626),
          (['de', 'sobre'], 3.3219280948873626),
          (['gustan', 'las'], 3.3219280948873626),
          (['gustan', 'los'], 3.3219280948873626),
          (['jugos', 'de'], 3.3219280948873626),
          (['me', 'gustan'], 3.3219280948873626),
          (['mí', 'no'], 3.3219280948873626),
          (['no', 'me'], 3.3219280948873626),
          (['sopas', 'de'], 3.3219280948873626)]
```

Contenido

- 1 Introducción
- 2 Modelos de Lenguaje
- 3 Estimación estadística de LM**
- 4 Evaluación de LMs
- 5 Tarea - Un primer modelo de lenguaje
- 6 Herramientas

- Para calcular la probabilidad de un *N-grama* tenemos que calcular entonces

$$P(w_i | w_{i-k} \dots w_{i-1}) = \frac{P(w_{i-k} \dots w_i)}{P(w_{i-k} \dots w_{i-1})}$$

- Conseguir un buen estimador radica simplemente en calcular las probabilidades anteriores.
- Veremos a continuación varias formas de calcularlas.

- El algoritmo MLE estima las probabilidades a partir de las frecuencias relativas

$$\begin{aligned} P(w_i | w_{i-k} \dots w_{i-1}) &= \frac{\text{conteo}(w_{i-k} \dots w_i)}{\text{conteo}(w_{i-k} \dots w_{i-1})} \\ &= \frac{C(w_{i-k} \dots w_i)}{C(w_{i-k} \dots w_{i-1})} \end{aligned}$$

- Estimando probabilidades de bigramas

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

- Ejemplo:

<s> Yo soy Pedro </s>

<s> Pedro es ingeniero </s>

<s> Yo nací en Pereira en el 89 </s>

$$P(\text{Yo}|\text{<s>}) = \frac{2}{3} = 0.67$$

$$P(\text{</s>}|\text{Pedro}) = \frac{1}{2} = 0.5$$

$$P(\text{Pereira}|\text{en}) = \frac{1}{2} = 0.5$$

- Veamos algunos ejemplos de la base de datos del *Berkeley Restaurant Project*¹
 - okay let's see i want to go to a thai restaurant with less than ten dollars per person
 - i want indian food and i have chuck's car so i can drive in
 - like to eat pork or beef
 - tell me about the thai barbecue place

¹<https://github.com/wooters/berp-trans>

Estimación de LMS - Maximum Likelihood Estimator

- En total hay 8566 frases

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Estimación de LMS - Maximum Likelihood Estimator

- Normalizando por número de unigramas

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Se obtiene la siguiente estimación

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

- Para calcular la probabilidad de una frase basada en la estimación de los bigramas

$$\begin{aligned} P(<s> \text{ I want english food } </s>) = \\ & P(\text{I} | <s>) \times \\ & P(\text{want} | \text{I}) \times \\ & P(\text{english} | \text{want}) \times \\ & P(\text{food} | \text{english}) \times \\ & P(</s> | \text{food}) \times \\ & = 0.000031 \end{aligned}$$

- Hay que tener en cuenta que:
 - Incluso si usamos un corpus extenso, los datos tendrán una dispersión muy alta.
 - Pocas palabras serán frecuentes, mientras que la inmensa mayoría no lo serán.
 - El algoritmo MLE asigna probabilidad cero a eventos no vistos (*unseen events*).
- Lo anterior significa que MLE nos puede dar problemas al querer estimar la probabilidad de una frase que contenga N-gramas inexistentes; la probabilidad cero de uno de los N-gramas hará que la probabilidad de toda la frase sea igual a cero.
- **Posible solución:** asignar una fracción de la **masa de probabilidad** a **eventos no vistos**.

Estimación de LM - Palabras *Out Of Vocabulary*

- En algunas tareas de PLN podemos suponer que conocemos todas las palabras que puedan ocurrir. Es decir, que se conoce el tamaño del vocabulario V .
- En tareas de este tipo, no tendremos palabras fuera del vocabulario (palabras **OOV** - *Out Of Vocabulary*).
- Sin embargo, en la mayoría de tareas **sí tendremos palabras que el sistema no conoce**, que no hemos visto en la etapa de entrenamiento o que simplemente **no existían** a la hora de crear nuestro modelo.
- Estos eventos no vistos (*unseen events*) son los que conocemos como palabras fuera de vocabulario (palabras **OOV**).
- El porcentaje de palabras OOV que aparecen en nuestros conjuntos de prueba se conoce como *OOV rate*.
- Las palabras OOV se etiquetan como <UNK>.

Estimación de LM - Palabras *Out Of Vocabulary*

- En algunas tareas de PLN podemos suponer que conocemos todas las palabras que puedan ocurrir. Es decir, que se conoce el tamaño del vocabulario V .
- En tareas de este tipo, no tendremos palabras fuera del vocabulario (palabras **OOV** - *Out Of Vocabulary*).
- Sin embargo, en la mayoría de tareas **sí tendremos palabras que el sistema no conoce**, que no hemos visto en la etapa de entrenamiento o que simplemente **no existían** a la hora de crear nuestro modelo.
- Estos eventos no vistos (*unseen events*) son los que conocemos como palabras fuera de vocabulario (palabras **OOV**).
- El porcentaje de palabras OOV que aparecen en nuestros conjuntos de prueba se conoce como *OOV rate*.
- Las palabras OOV se etiquetan como <UNK>.

Estimación de LMS - Laplace Smoothing

Laplace Smoothing (Add-one smoothing)

Cuando se tienen datos dispersos

$P(w|\text{denied the})$

3 allegations

2 reports

1 claims

1 request

“Robar” probabilidad $\rightarrow$ suaviza estimaciones

$P(w|\text{denied the})$

2.5 allegations

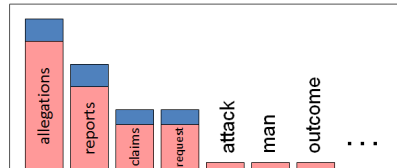
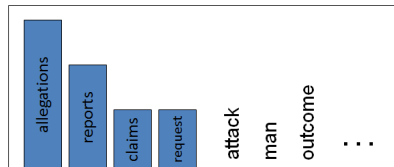
1.5 reports

0.5 claims

0.5 request

2 otros

7 total



Estimación de LMS - Laplace Smoothing

- El suavizado de Laplace se conoce también como suavizado *Add-One* ya que básicamente consiste en pretender que cada palabra ha aparecido una vez más.
- Tan sencillo como **sumar uno** a todas las cuentas.

$$P_{MLE}(w_i|w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$
$$P_{Lap}(w_i|w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$$

donde V es el tamaño del vocabulario

Estimación de LMS - Laplace Smoothing

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Estimación de LMS - Laplace Smoothing

Veamos el conteo recalculado

$$C^*(w_{i-1}, w_i) = \frac{[C(w_{i-1}, w_i) + 1] \times C(w_{i-1})}{C(w_{i-1}) + V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Estimación de LMS - Laplace Smoothing

Veamos los bigramas recalculados

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Estimación de LMS - Laplace Smoothing

- El método de suavizado de Laplace no se acostumbra usar en la estimación de n -gramas para tareas de reconocimiento de habla.
- En casos donde el número de eventos no vistos sea demasiado alto, el suavizado de Laplace puede llegar a destinar hasta aproximadamente el 99.9% de la masa de probabilidad a estos eventos, dejando sin peso a los eventos que sí ocurren en la base de datos.
- Este suavizado es muy empleado en tareas de clasificación de textos.
- Y principalmente en dominios de aplicación donde el **número de ceros no es muy grande**.

- En muchas ocasiones en las que no se puede dar mucha confianza a la estimación, conviene usar menos memoria; es decir, “*echar para atrás*” y emplear menos información contextual.
- El método de **Backoff** básicamente establece que si se tiene una buena evidencia se pueden emplear **trigramas**, de lo contrario **bigramas**; o de lo contrario **unigramas**.
- De esta forma se hará **backoff** (*marcha atrás*) hasta encontrar el modelo con la historia más confiable para conseguir mejores resultados.

- El método de **backoff** plantea lo siguiente:

$$P_{\text{backoff}}(w|h) = \begin{cases} P(w|h) & C(h, w) > 0 \\ \alpha(h)P_{\text{backoff}}(w|h^-) & \text{de lo contrario} \end{cases}$$

donde h^- es la historia modificada, es decir, donde se elimina la palabra más antigua; y $\alpha(h)$ es el peso de **backoff** calculado como

$$\alpha(h) = \frac{1 - \sum_{w: C(h,w) > 0} P^*(w|h)}{1 - \sum_{w: C(h,w) > 0} P^*(w|h^-)}$$

- Problemas que pueden presentarse con el método de **backoff**:
 - Supongamos que el bigrama $\langle a, b \rangle$ y el unigrama $\langle c \rangle$ son muy comunes, pero el trigramma $\langle a, b, c \rangle$ no aparece nunca (bien sea porque no está en el corpus o porque su construcción no esté permitida dentro del idioma).
 - En vez de asignar un valor de cero a $P(c|a, b)$, este método hará **backoff** hasta el bigrama y estimará entonces $P(c|b)$, la cual podría ser muy alta.

- Una forma de solucionar la dispersión en un n-grama de orden superior es “mezclar” ese modelo con modelos de orden inferior (bigramas, unigramas) que sufran menos dispersión.
- La *interpolación lineal* surge como una forma de combinar la información a distintos niveles. A los modelos que resultan de este método también se les conoce como **mixture models**.
- Por ejemplo, el caso más común de interpolación es en la obtención de trigramas

$$P_{li}(w_i|w_{i-1}, w_{i-2}) = \lambda_1 P_1(w_i) + \lambda_2 P_2(w_i|w_{i-1}) + \lambda_3 P_3(w_i|w_{i-1}, w_{i-2})$$

donde $0 \leq \lambda_i \leq 1$ y $\sum_i \lambda_i = 1$

- ¿Cómo establecer el valor de los λ_i ?
 - Empleando un corpus de validación (*held-out corpus*)



- Se deben seleccionar los λ_i que maximicen la probabilidad sobre los datos de validación.
- Previamente se deben de haber calculado los n-gramas sobre el conjunto de entrenamiento.

Tarea: Investigar el método de suavizado de Kneser-Ney.

- 1 Introducción
- 2 Modelos de Lenguaje
- 3 Estimación estadística de LM
- 4 Evaluación de LMs**
- 5 Tarea - Un primer modelo de lenguaje
- 6 Herramientas

Evaluación de modelos de lenguaje (1)

- ¿Qué tan bueno es nuestro modelo?
 - ¿Nuestro modelo asigna probabilidad alta a frases “buenas” o a “malas”?
- El modelo debe entrenarse sobre un conjunto de **entrenamiento** y evaluarse sobre un conjunto de **evaluación o de test**.
- Debemos emplear una métrica de evaluación que nos permita discernir acerca de la calidad del modelo.

- **Evaluación extrínseca**

- Entrenamos varios modelos (digamos modelos A y B).
- Los sometemos a una tarea común (p.ej. una tarea de corrección gramatical, de traducción automática, etc.)
- Medimos la precisión de cada modelo como la cantidad de palabras equivocadas que el modelo corrigió, o como la cantidad de palabras que tradujo correctamente.
- Comparamos la precisión de ambos modelos.

- **Evaluación intrínseca**

- **Perplejidad:** La perplejidad (PP) es una medida de la incertidumbre de las predicciones.
- Básicamente es una forma de medir qué tan bien el modelo puede predecir la siguiente palabra (juego de Shannon).
- Un buen modelo es aquel que asigne una alta probabilidad a la palabra que efectivamente ocurre.

- **Evaluación intrínseca**

- La PP está definida como

$$\begin{aligned} PP(W) &= P(w_1 w_2 \dots w_n)^{1/n} \\ &= \sqrt[n]{\frac{1}{P(w_1 w_2 \dots w_n)}} \\ &= \sqrt[n]{\prod_{i=1}^n \frac{1}{P(w_i | w_1 \dots w_{i-1})}} \quad \text{Por regla de cadena} \\ &= \sqrt[n]{\prod_{i=1}^n \frac{1}{P(w_i | w_{i-1})}} \quad \text{Para bigramas} \end{aligned}$$

- Minimizar la perplejidad equivale a maximizar la probabilidad.

- En todo este tema del día de hoy se suele trabajar en espacio logarítmico
 - Se evita el *underflow* en los cálculos matemáticos.
 - Computacionalmente sumar es más rápido que multiplicar

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

Cómo entrenar modelos de lenguaje (ejemplo en SRILM)

Speech Technology and Research Laboratory - SRILM toolkit

(**Nota: Ver instrucciones que tengo en el MAC**)

<http://www.speech.sri.com/projects/srilm/>

Para entrenar un modelo de lenguaje

```
ngm-count -text TRAINDATA -lm LM
```

```
ngm-count -order 2 -text texto_entrada.txt
```

```
-write-vocab wordlist -lm LM_salida
```

Miremos la importancia de las OOV en la estimación de los LM.

```
ngm-count -order 2 -unk -text texto_entrada.txt
```

```
-write-vocab wordlist -lm LM_salida
```

Veamos qué significa cada parámetro en el modelo resultante.

Para generar textos aleatorios (ejemplo en SRILM)

Speech Technology and Research Laboratory - SRILM toolkit

<http://www.speech.sri.com/projects/srilm/>

Para generar textos a partir de un LM

```
-ngram -gen N -lm LM_file
```

Veamos la coherencia de los textos.

Para evaluar modelos de lenguaje (ejemplo en SRILM)

Speech Technology and Research Laboratory - SRILM toolkit

<http://www.speech.sri.com/projects/srilm/>

Para calcular la perplejidad de un modelo

```
ngm -lm LM_file -ppl TEST_DATA -debug 2
```

Analizamos los resultados de la perplejidad **con** OOV y **sin** OOV.

Contenido

- 1 Introducción
- 2 Modelos de Lenguaje
- 3 Estimación estadística de LM
- 4 Evaluación de LMs
- 5 Tarea - Un primer modelo de lenguaje**
- 6 Herramientas

Tarea - Nuestro primer modelo de lenguaje

- Sobre un corpus extenso (compuesto por algunos documentos de miles de líneas de extensión):
 - Implementar un tokenizador.
 - Extraer el vocabulario.
 - Calcular los unigramas.
 - Calcular los bigramas.
 - A partir de las probabilidades de los bigramas, generar de forma automática frases (entregar como mínimo 100 frases generadas de forma automática).

Contenido

- 1 Introducción
- 2 Modelos de Lenguaje
- 3 Estimación estadística de LM
- 4 Evaluación de LMs
- 5 Tarea - Un primer modelo de lenguaje
- 6 Herramientas**

- Natural Language Toolkit (python) <http://www.nltk.org/>
- Hidden Markov Models Toolkit (C) <http://htk.eng.cam.ac.uk/>
- Statistical Machine Translation <http://www.statmt.org/moses/>

- The SRI Language Modeling Toolkit <http://www.speech.sri.com/projects/srilm/>
- KenLM Language Model Toolkit <https://kheafield.com/code/kenlm/>
- Google n-gram book viewer <https://books.google.com/ngrams>
- Info Google n-gram book viewer <https://books.google.com/ngrams/info>
- The F* word <http://english.stackexchange.com/questions/327087/the-f-word-in-n-gram-viewer>
- How the Internet talks <https://projects.fivethirtyeight.com/reddit-ngram/>

Referencias I



Manning, C. and Schütze, H. (2001).
Foundations of Statistical Natural Language Processing.
The MIT Press. 1st Edition. 2001



Jurafsky, D. and Martin, J. (2008).
Speech and Language Processing
Ed. Pearson. 2008